Entwicklerdokumentation: Architektur

Diese Dokumentation beschreibt den technischen Aufbau der Anwendung Study Planner. Sie richtet sich an Entwickler, die den Code verstehen, erweitern oder warten möchten.

Architekturüberblick

Die Anwendung ist als lokale Java-Desktop-Anwendung umgesetzt.

Es gibt drei zentrale Schichten:

- UI
- Service
- Persistence

Die Datenobjekte liegen separat im Package `model`.

Paketstruktur

```
studyplanner/
|-- Main.java
|-- model/
|-- service/
|-- persistence/
|-- ui/
```

studyplanner

Dieses Package enthält den Einstiegspunkt der Anwendung.

Main

Main startet die Anwendung. Die Klasse:

- setzt das Swing Look and Feel
- erstellt Storage
- erstellt StudyService
- öffnet das Hauptfenster `StudyPlannerFrame`

studyplanner.model

Das Package `model` enthält die fachlichen Datenklassen.

Enthaltene Klassen:

- Modul
- Termin
- Lernblock
- Erinnerung
- Einstellung

Diese Klassen enthalten Validierungen für ihre eigenen Daten. Dadurch wird verhindert, dass ungültige Objekte entstehen.

Beispiele:

- Ein Modulname darf nicht leer sein.
- Ein Termin braucht ein Modul, ein Datum und eine Uhrzeit.
- Ein Lernblock braucht eine Dauer zwischen 1 und 1440 Minuten.

Eine Erinnerung braucht ein Ziel und einen gültigen Vorlauf.

studyplanner.service

Das Package `service` enthält die Anwendungslogik.

Enthaltene Klassen:

- StudyService
- StudyPlan

StudyService

`StudyService` ist die zentrale Schnittstelle für die Oberfläche. Die UI greift nicht direkt auf die Speicherdatei zu, sondern verwendet Methoden aus dem Service.

Der Service ist verantwortlich für:

- Module anlegen, bearbeiten und löschen
- Termine anlegen, bearbeiten und löschen
- Lernblöcke anlegen, bearbeiten und löschen
- Status von Terminen und Lernblöcken verwalten und Einträge nach Status filtern
- Erinnerungen erzeugen und aktualisieren
- Kalender-Einträge filtern
- fällige Erinnerungen berechnen
- Einstellungen speichern

Wichtige Designentscheidung:

Die UI enthält möglichst wenig Fachlogik. Fachliche Regeln liegen im `StudyService`.

StudyPlan

`StudyPlan` ist der zentrale Datencontainer der Anwendung.

Er enthält:

- Liste aller Module
- Liste aller Termine
- Liste aller Lernblöcke
- Liste aller Erinnerungen
- globale Einstellungen

Dieser Container wird komplett gespeichert und geladen.

studyplanner.persistence

Das Package `persistence` enthält die lokale Speicherung.

Storage

Storage speichert und lädt einen kompletten `StudyPlan`.

Die Anwendung nutzt keine externe Datenbank. Stattdessen wird der gesamte Datenbestand lokal serialisiert.

Standardpfad:

`<Benutzerordner>/.studyplanner/studyplanner.dat`

Designentscheidung:

Für diese Anwendung ist eine Datei-basierte Speicherung ausreichend, weil:

kein Mehrbenutzerbetrieb vorgesehen ist
keine Server-Infrastruktur benötigt wird
die Datenmenge klein ist
die Anwendung lokal und einfach bleiben soll

studyplanner.ui

Das Package `ui` enthält die Swing-Oberfläche.

StudyPlannerFrame

`StudyPlannerFrame` ist das Hauptfenster der Anwendung.

In den Bereichen Termine und Lernblöcke werden die Einträge tabellarisch dargestellt. Der Status eines Eintrags wird in einer eigenen Spalte angezeigt. Zusätzlich kann die Anzeige über Schaltflächen im unteren Bereich nach offenem oder erledigtem Status gefiltert werden.

Die Oberfläche enthält:

obere Titelleiste
linke Navigation
Modulübersicht
Kalenderansicht
Terminverwaltung
Lernblockverwaltung
Erinnerungsübersicht
Einstellungen

Die UI ruft Methoden des `StudyService` auf und aktualisiert danach ihre Tabellen oder Kartenansichten.

Datenmodell

Die Anwendung verwendet kein relationales Datenbankschema. Stattdessen wird ein Objektmodell gespeichert.

Das gespeicherte Datenmodell sieht vereinfacht so aus:

```
StudyPlan
|-- List<Modul>
|-- List<Termin>
|-- List<Lernblock>
|-- List<Erinnerung>
|-- Einstellung
```

Beziehungen zwischen den Daten

Modul

Ein Modul ist ein Studienfach oder Kurs.

Wichtige Felder:

id
name
farbe
semester
notizen

Termin

Ein Termin gehört zu einem Modul.

Wichtige Felder:

- id
- modulId
- titel
- datum
- uhrzeit
- notizen
- status

Die Verbindung zum Modul erfolgt über `modulId`.

Der Status legt fest, ob der Termin als offen oder erledigt behandelt wird.

Lernblock

Ein Lernblock gehört ebenfalls zu einem Modul.

Wichtige Felder:

- id
- modulId
- thema
- datum
- startzeit
- dauerMinuten
- status

Die Verbindung zum Modul erfolgt über `modulId`.

Der Status legt fest, ob der Lernblock als offen oder erledigt behandelt wird.

Erinnerung

Eine Erinnerung verweist auf einen Termin oder Lernblock.

Wichtige Felder:

- id
- zielTyp
- zielId
- vorlaufMinuten
- aktiv
- zuletztAngezeigt

`zielTyp` legt fest, ob die Erinnerung zu einem Termin oder Lernblock gehört. `zielId` enthält die ID des Zielobjekts.

Einstellung

Einstellung enthält globale Konfigurationen.

Wichtige Felder:

- standardVorlaufMinuten
- erinnerungenAktiv

Objektbeziehungen

```
Modul 1 -- n Termin
Modul 1 -- n Lernblock
Termin 1 -- 0..1 Erinnerung
Lernblock 1 -- 0..1 Erinnerung
StudyPlan 1 -- 1 Einstellung
```

Persistenz statt Datenbank

Es gibt kein SQL-Schema und keine Tabellen.

Stattdessen gilt:

- `StudyPlan` entspricht dem gesamten Datenbestand.
- Listen in `StudyPlan` entsprechen Tabellen.
- IDs verbinden die Objekte miteinander.
- `Storage` schreibt und liest den gesamten `StudyPlan`.

Vergleich zu einem relationalen Schema:

```
Module(id, name, farbe, semester, notizen)
Termine(id, modulId, titel, datum, uhrzeit, notizen)
Lernblöcke(id, modulId, thema, datum, startzeit, dauerMinuten)
Erinnerungen(id, zielTyp, zielId, vorlaufMinuten, aktiv, zuletztAngezeigt)
Einstellungen(standardVorlaufMinuten, erinnerungenAktiv)
```

Dies ist nur ein konzeptionelles Schema. Es wird nicht als echte Datenbank umgesetzt.

Wichtige Designentscheidungen

Java Swing

Die Anwendung nutzt Swing, weil:

- Swing Teil der Java-Standardbibliothek ist
- keine externen UI-Abhängigkeiten benötigt werden
- eine einfache Desktop-Anwendung schnell umgesetzt werden kann

Maven

Maven wird genutzt für:

- Projektstruktur
- Build
- Tests
- Javadoc
- Abhängigkeitsverwaltung

JUnit 5

JUnit 5 wird für automatisierte Tests genutzt.

Getestet werden:

- Model-Klassen
- Persistence
- Service-Logik

Die Swing-Oberfläche wird aktuell nicht automatisiert getestet, weil UI-Tests aufwendiger sind und der größte fachliche Nutzen in den Service- und Model-Tests liegt.

Lokale Datei statt Datenbank

Eine lokale Datei wurde gewählt, weil die Anforderungen keine Datenbank benötigen.

Vorteile:

- einfache Installation
- kein Datenbankserver
- keine Zugangsdaten
- keine Migrationen
- passend für Einzelbenutzerbetrieb

Nachteile:

- nicht für parallelen Mehrbenutzerbetrieb geeignet
- kein direktes SQL-Querying
- bei größeren Datenmengen wäre eine Datenbank sinnvoller

Fehlerbehandlung

Ungültige Eingaben werfen im Model oder Service eine `IllegalArgumentException`.

Die UI fängt diese Exceptions ab und zeigt die Fehlermeldung als Dialog an.

Beim Laden einer beschädigten Speicherdatei startet die Anwendung mit einem leeren `StudyPlan`, statt abzustürzen.

Teststrategie

Die Tests liegen unter:

```
src/test/java
```

Aktuell gibt es Tests für:

- model
- persistence
- service

Tests werden ausgeführt mit:

```
mvn test
```

Erweiterungshinweise

Neue Features sollten möglichst dieser Struktur folgen:

1. Neue Daten zuerst im `model` ergänzen.
2. Fachlogik im `service` implementieren.
3. Speicherung über `StudyPlan` und `Storage` sicherstellen.
4. UI in `StudyPlannerFrame` erweitern.
5. Tests für Model, Service oder Persistence ergänzen.

Die UI sollte nicht direkt mit `Storage` arbeiten. Der Zugriff auf Daten soll über `StudyService` erfolgen.